SECURITY AUDIT

NAMI

Farming

FINAL REPORT

Oct 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	NAMI – Farming – Audit Report
Website	nami.fi
Language	Solidity
Methods	Manual Analysis
Github repository	base commit: https://github.com/cryptoalgebra/Algebra/tree/62f01348af24b96d7d16f6b56fbaa61e512ba1c5/src/farming audit commit: https://github.com/namifi/Algebra-Farming/blob/9485dc1cd620b7053ec38a1afe976a2fa4d2b92e/src/farming/contracts/farmings
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium						
Low	7			7		
Informational						
Governance						
Total	7			7		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

FarmingCenter - Differential

The following changes have been made:

- Contract was made upgradeable
- _updatePosition function was refactored to align with the new position adjustment behavior
- collectRewards includes forfeited return value

The contract was essentially adjusted to be compatible with the new approach within the AlgebraEternalFarmingUpgradeable contract.

https://www.diffchecker.com/MuD6nKOL/

No issues found.

AlgebraEternalFarmingUpgradeable - Differential

The following changes have been made:

- Contract was made upgradeable
- Farm struct includes enteredTimestamp and pendingRewards information
- Storage support for emissionsToken, claimer and buffer feature
- Implement admin collectForfeitedRewards function
- Implement setters for claimer, emissionToken, farmingBuffer and buffer exclusion
- Refactor liquidity delta application from enter-and-exit to in-place adjustment
- General implementation of buffer, carry and forfeit mechanism

Essentially, the following intentions are behind these changes:

- Implementation of a buffer mechanism to counter JIT liquidity provision
- Incorporation of Claimer mechanism

https://www.diffchecker.com/mOV7c5hV/

Issue_01	Additional edge-cases from emergency situations / governance interventions
Severity	Low
Description	Throughout the report, we have already highlighted specific edge-cases from emergency situations and/or governance executions. We will furthermore aggregate additional situations here: • An emergency exit deletes the farm without removing its liquidity from the virtual pool, and no AEF path can remove it later. While the price sits in the ghost range, every distribution divides the stream by ghost-inclusive liquidity, so live farmers lose the ghost share (all of it in ghost-only windows), which stays in AEF under no ledger. • Deleting a farm can leave its liquidity in the virtual pool: the emergency branch of AlgebraEternalFarmingUpgradeable.exitFarming skips the virtual-pool update, and on a deactivated pool AlgebraEternalFarmingUpgradeable._updatePosition sends a removal the pool ignores. Later streams stay divided by that phantom liquidity, so live farmers are diluted and its share of the funder's reserve is stranded. • The virtual pool's liquidity should equal the live in-range farms' sum, but AEF breaks it three ways: an emergency exit deletes the farm without removing its liquidity, exits after auto-deactivation are dropped, and an entry whose own poke auto-deactivates the pool is recorded uncounted. Live farmers lose stream share to ghost weight, or other claimants fund an uncounted farm. • After the NFPM farming center is re-pointed, an old-center farmed NFT can be drained and burned without any farming hook. FarmingCenterUpgradeable's exit and collect then revert in isApprovedOrOwner and AlgebraEternalFarmingUpgradeable.exitFarming admits only the FarmingCenter, so the farm's virtual-pool weight persists until a privileged rescue, diluting honest farmers while its share accrues

	uncollectable. • AlgebraEternalFarmingUpgradeable._enterFarming checks deactivation only before its virtual-pool poke; when that poke auto-deactivates a tick-desynced virtual pool it adds no liquidity, yet enterFarming still records the farm. Collects then credit growth on liquidity absent from currentLiquidity, never forfeited and paid from the commingled reward-token balance, so other incentives' reserves and users' credited rewards go unpaid. • AlgebraEternalFarmingUpgradeable.setFarmingCenterAddress re-points farmingCenter with no live-deposit check or hand-over. Farms entered through the old center then fail onlyFarmingCenter, and the new center rejects them at its deposits check. While the position manager still hooks the old center, farmed LPs cannot change liquidity or exit; once it is re-pointed, stranded farms keep collecting on stale liquidity, diluting honest farmers.
Recommendations	Consider acknowledging these issues.
Comments / Resolution	Acknowledged.

Issue_02	Exit uses a stale forfeiture decision when the virtual pool deactivates during settlement
Severity	Low
Description	AlgebraEternalFarmingUpgradeable.exitFarming() determines whether rewards should be forfeited before calling _updatePosition(). However, the subsequent call to EternalVirtualPool.applyLiquidityDeltaToPosition() can detect a tick desynchronization, deactivate the virtual pool, and return successfully. _updatePosition() does not reevaluate the incentive's status afterward and instead settles the rewards using the previously calculated forfeiture decision. Consequently, a non-exempt farmer exiting within the farming buffer can have all unsettled rewards, including pending carry, credited to forfeitedRewards even though the incentive deactivated during that exit. The farm is subsequently deleted, preventing the farmer from recovering that entitlement through normal settlement. This also creates inconsistent treatment between exit routes: updateFarming() explicitly rechecks deactivation after updating the virtual pool and disables forfeiture when closing a deactivated farm, whereas an explicit exit or complete liquidity withdrawal retains the stale decision.
Recommendations	Consider acknowledging this behavior due to the theoretical likelihood of virtual pool deactivation.
Comments / Resolution	Acknowledged.

Issue_03	Emergency withdrawals discard unsettled farming rewards
Severity	Low
Description	When isEmergencyWithdrawActivated is enabled, AlgebraEternalFarmingUpgradeable.exitFarming() bypasses normal reward settlement. It only credits farm.pendingReward0 and farm.pendingReward1 to the owner's reward balances before deleting the farm. Rewards represented by the difference between current virtual-pool reward growth and the farm's stored growth snapshots are not calculated or credited. A position can therefore have substantial accrued rewards while both pending fields remain zero, since those fields contain carried rewards from earlier buffer-gated updates rather than the farm's entire outstanding entitlement. An emergency exit permanently removes the snapshots needed to settle the remaining accrual, including rewards earned before emergency mode was activated. This also affects ordinary NFPM liquidity changes during emergency mode because FarmingCenter routes them into the same emergency exit path. Previously credited owner rewards remain intact; the loss concerns rewards still represented by the deleted farm's growth entitlement.
Recommendations	Since this issue is inherited from the Algebra behavior, we recommend acknowledging it.
Comments / Resolution	Acknowledged.

Issue_04	Farming buffer change is impactful in hindsight
Severity	**Low**
Description	The farming contract stores enteredTimestamp for each farm but does not snapshot the applicable farming buffer. Instead, _shouldForfeitRewards() reads the current poolFarmingBuffer or defaultFarmingBuffer whenever rewards are collected or settled. Changes made through setFarmingBuffer() therefore immediately affect existing positions rather than applying exclusively to subsequent entries or liquidity increases. As a result, increasing the buffer can cause a previously mature position to become buffer-gated again. Rewards that were already eligible for collection, but had not yet been credited to the owner, can subsequently be withheld or forfeited upon a decrease or exit. Pending carry is subject to the same reevaluation. Conversely, reducing the buffer can immediately release previously gated rewards. Changes to the default buffer affect every pool whose override is zero. This does not claw back rewards already recorded in rewards[owner][token]; it changes the treatment of existing, unsettled farm entitlements.
Recommendations	Consider communicating each buffer change with the community upfront.
Comments / Resolution	Acknowledged.

Issue_05	Entry can register a farm without registering its virtual liquidity
Severity	**Low**
Description	AlgebraEternalFarmingUpgradeable._enterFarming() checks whether the incentive is deactivated before adding the position's liquidity to the virtual pool. However, EternalVirtualPool.applyLiquidityDeltaToPosition() can itself detect desynchronization, set deactivated = true, and return without applying that liquidity addition. The entry flow does not recheck deactivation or confirm that the virtual liquidity was registered before proceeding. **This entry can complete when both position boundary ticks are already initialized in the virtual pool.** In that case, the subsequent reward-growth lookup succeeds, and enterFarming() stores a new Farm with positive liquidity even though that liquidity was never added to the virtual pool. If either boundary tick is uninitialized, the lookup instead reverts and rolls back the entry. The successful case creates a mismatch between the recorded farm liquidity and the virtual pool's distribution accounting. Where nonzero reward rates continue and the position's range receives growth under the frozen virtual state, subsequent collection can credit rewards for liquidity that was never included in the distribution denominator. This can over-allocate rewards relative to the emissions actually distributed by the virtual pool.
Recommendations	A solution would be to re-check deactivation immediately after the virtual-pool call in _enterFarming(). However, this would also touch existing Algebra logic instead of the diff and therefore we recommend acknowledging this.
Comments / Resolution	Acknowledged.

Issue_06	Side-effects from farmingCenter rotation
Severity	**Low**
Description	NonfungiblePositionManager.setFarmingCenter() changes the globally configured FarmingCenter without migrating existing tokenFarmedIn[tokenId] associations. However, _applyLiquidityDeltaInFarming() only forwards a liquidity update when the position's recorded FarmingCenter matches the currently configured address. Rotating to a different FarmingCenter therefore causes subsequent liquidity changes on existing farmed positions to silently skip their original farming callback. Where the old incentive remains active and collection through its original backend remains available, users can reduce or completely withdraw their real AMM liquidity while farming continues calculating rewards using the previous Farm.liquidity. This permits emissions to be allocated to positions that no longer provide the corresponding liquidity. If the emptied NFT is subsequently burned, its farming records can remain without an existing NFT owner able to invoke the ordinary ownership-gated exit, leaving orphaned virtual liquidity.
Recommendations	Consider not rotating the FarmingCenter address.
Comments / Resolution	Acknowledged.

Issue_07	Recovering rewards after automatic deactivation can finalize distributions to already-deleted farms
Severity	Low
Description	Automatic deactivation of EternalVirtualPool does not zero its reward rates, and subsequent liquidity-removal calls return without updating its virtual liquidity. Meanwhile, EternalFarming can settle and delete the corresponding farm records. This permits a deactivated virtual pool to retain positive currentLiquidity, nonzero rates, and remaining reserves after all associated farms have closed. A later decreaseRewardsAmount() call distributes rewards before reading the reserves and calculating the recoverable amount. Since _distributeRewards() does not check deactivation, elapsed emissions are deducted from the remaining reserves and assigned to growth calculated against the orphaned virtual liquidity. No surviving farm can settle that growth, and the deducted amount is not credited to either owner rewards or forfeitedRewards. The recovery transaction can therefore succeed while returning substantially less than the previously recorded reserve, leaving the difference physically held by EternalFarming but unrepresented by its supplied accounting-based withdrawal paths. This affects both reward sides because distribution runs for both tokens even when only one is being withdrawn. Delayed manual deactivation does not restore the lost amount: its rate-zeroing call first distributes at the previous rates and only afterward stops future emissions.
Recommendations	Consider acknowledging this behavior, a fix would touch code which is unrelated to the audit scope.
Comments / Resolution	Acknowledged.